Boku-nano

限られた日本語指示から
Python関数を生成する小型言語モデル



本発表の概要

> 制限された日本語指示からPython関数を生成する小型言語モデルをゼロから構築し
> 合成データのみの学習でChatgpt 6 Astraに匹敵する精度の達成

目的

学部生が1人でフロンティアモデルに匹敵する小型モデルの作成方法が分かる

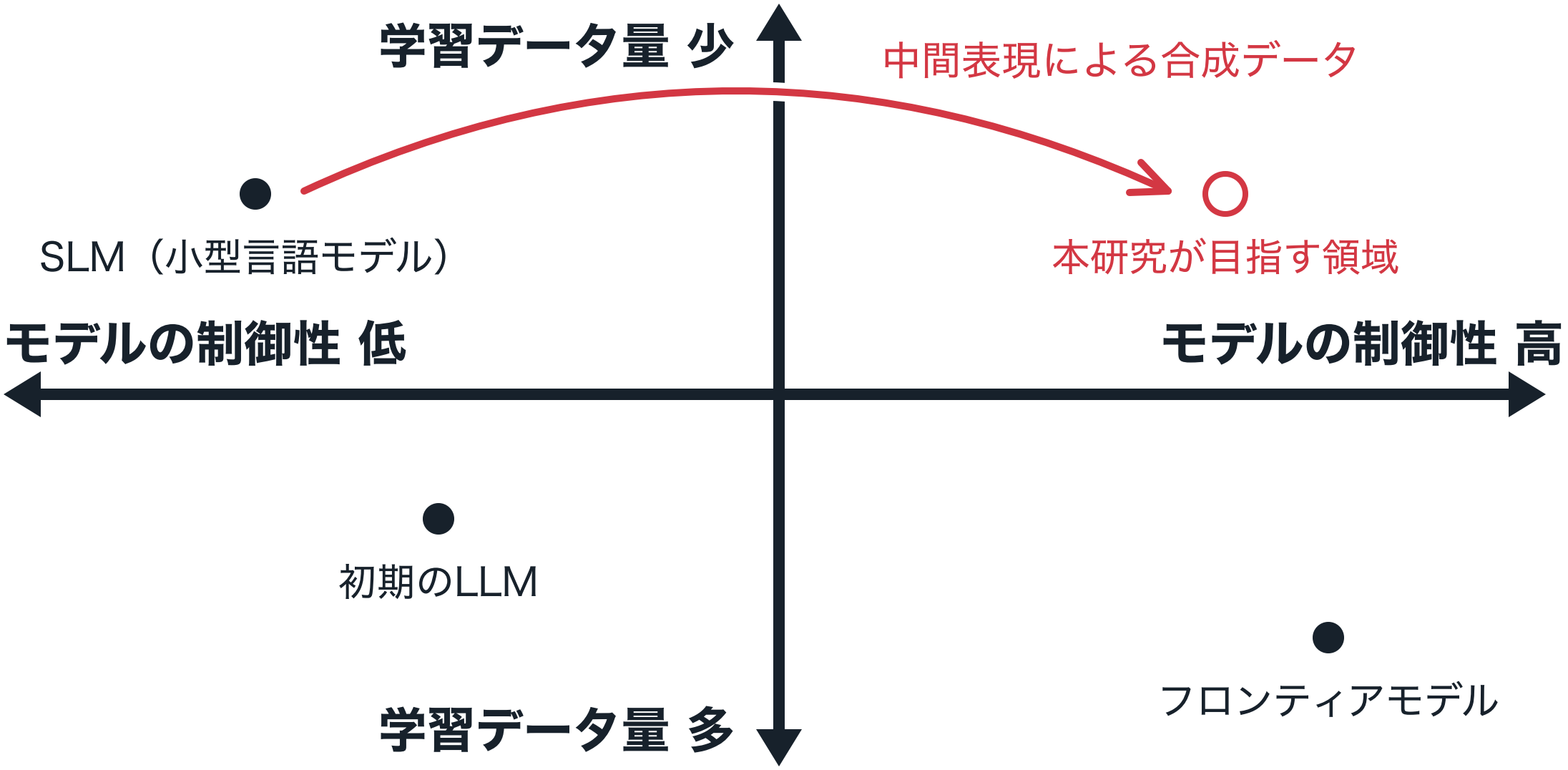
特徴

中間表現により生成した合成データの質で大規模計算に対抗

利点

- 約1Mのモデル・5090で1分で訓練可能
- 訓練データの制御性：文の形・操作の数（難しさ・文の長さ）・使う語彙の数の制御が可能
- 応用範囲：関数で書ける処理なら同じ方法でデータ合成からモデル学習まで全て可能

背景：モデルの制御性と学習データ量による位置づけ



SLMの発展

分野①　自然言語そのもの（物語）

TinyStories
GPT-3.5/4による短い物語の合成
3〜4歳児が分かる語彙のみの使用
1.25MのTransformerでも一貫した文章生成

SimpleStories
平易な語彙のみの使用
話題・文体・文法・書き出しによるデータ多様性向上
より少ないパラメータでLLM評価の得点が向上

WRAP
Webの文章の4文体への言い換えと350M・1.3Bの事前学習
同じ性能に約3倍速く到達／perplexityが10%以上改善

文章の良さに対する人やLLMの評価
人の好みを用いた学習：**RLHF**
LLMによる採点：**LLM-as-a-Judge**

分野②　code・math

phi-1：code
教科書品質の選別データとGPT-3.5で合成した教科書・演習で1.3Bモデルの学習
約7Bトークンの学習でHumanEval 50.6%

TinyGSM：math
GPT-3.5で合成した約1,230万件の算数問題とPython解答で1.3Bモデルの学習
検証器と組み合わせGSM8K 81.5%

テストや正解照合で正誤の検証
検証結果を報酬とした学習
RLVR（検証可能な報酬による強化学習）

意味ASTによる意味の固定と言い換えの生成

共通の処理の定義：意味AST

```
{"sequence": [
    {"filter": ["even"]}, {"map": ["add_k"]}
]}
```

同じ意味を持つ日本語の言い換え

生成の手順

Qwen3-4B-AWQで各操作の日本語表現生成
承認済みの表現をASTの順に組み合わせ
同じ処理を指す指示文生成

生成例

「偶数だけを残し、各要素にkを加える」

「偶数を選び出して、各要素にkを足す」

検証済みのPythonコード

生成の手順

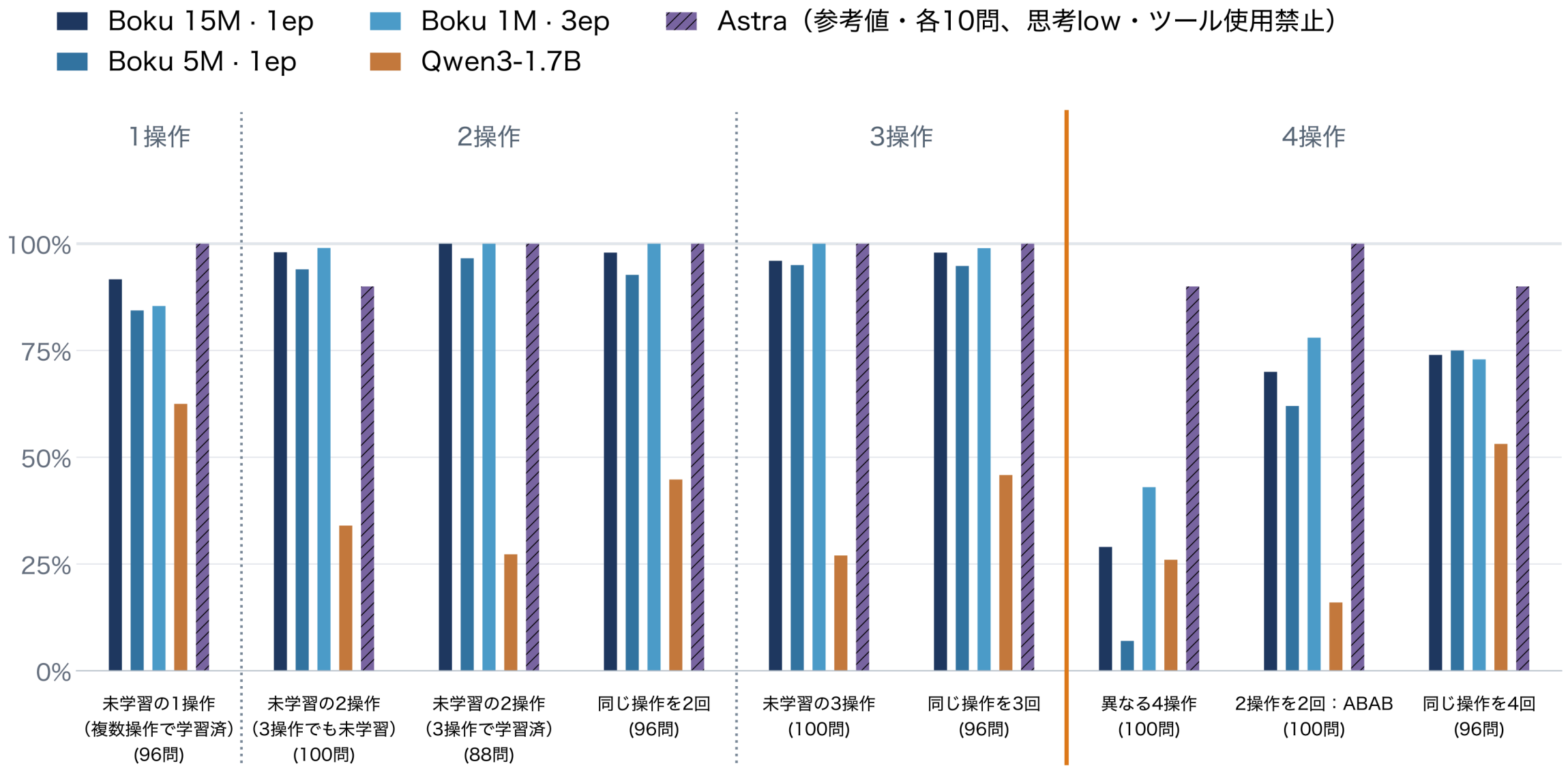
ASTの操作からPythonコードへの変換
構文の確認と
参照インタプリタとの実行結果の照合

生成例

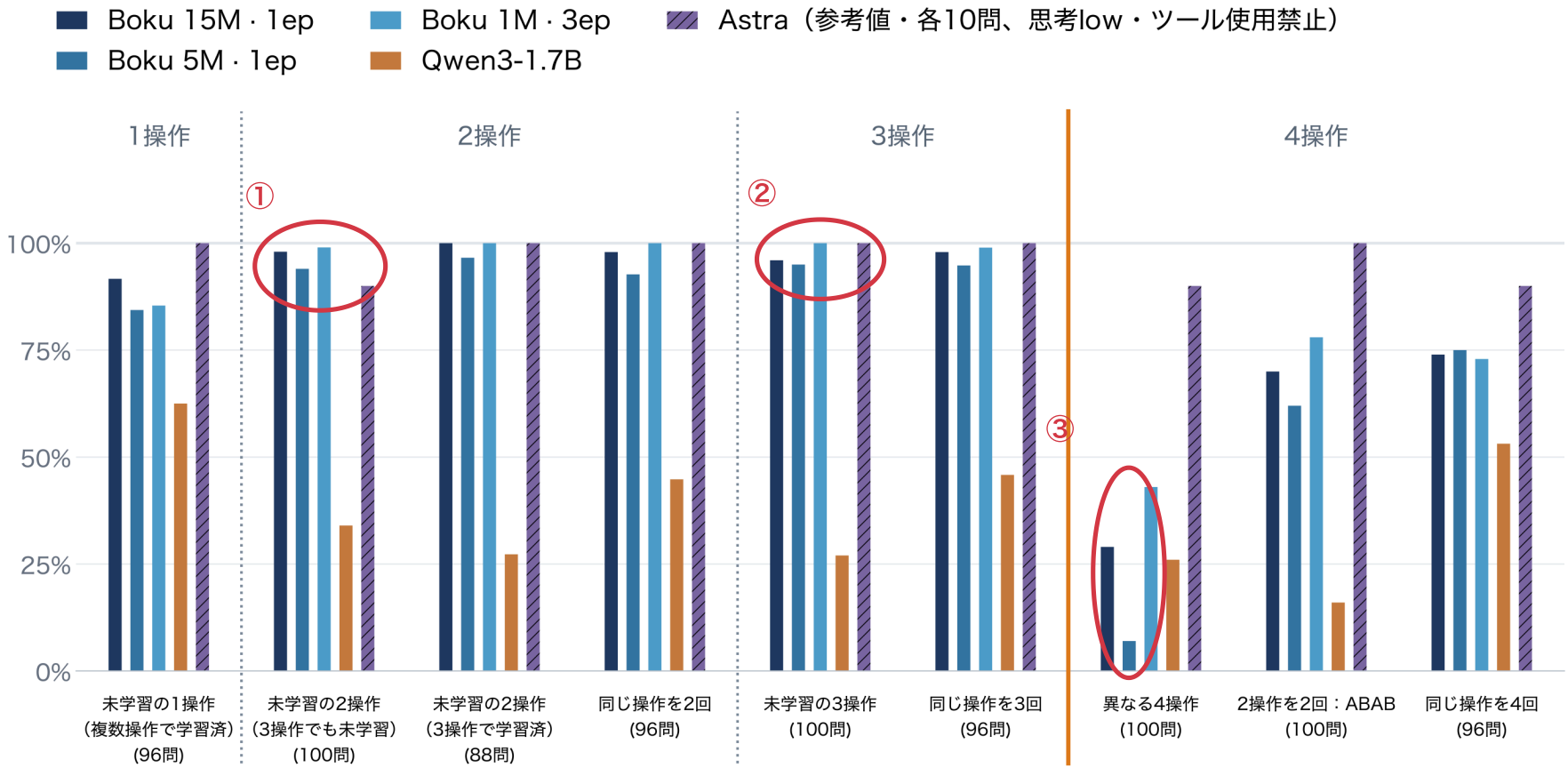
```
def solve(xs, k):
        return [x + k for x in xs
                      if x % 2 == 0]
```

共通の中間表現による意味の対応を保った合成データの拡充
意味の保持を重視した言い換えによるデータ効率の改善：SynPro（Yu & Xiong, 2026） の報告

独自ベンチマークによる評価結果



評価結果の注目点3か所



① 未学習の2操作
1M・3 epoch：99%
Astra参考値：90%

② 未学習の3操作
1M・3 epoch：100%
学習済みの操作による新しい組み合わせ

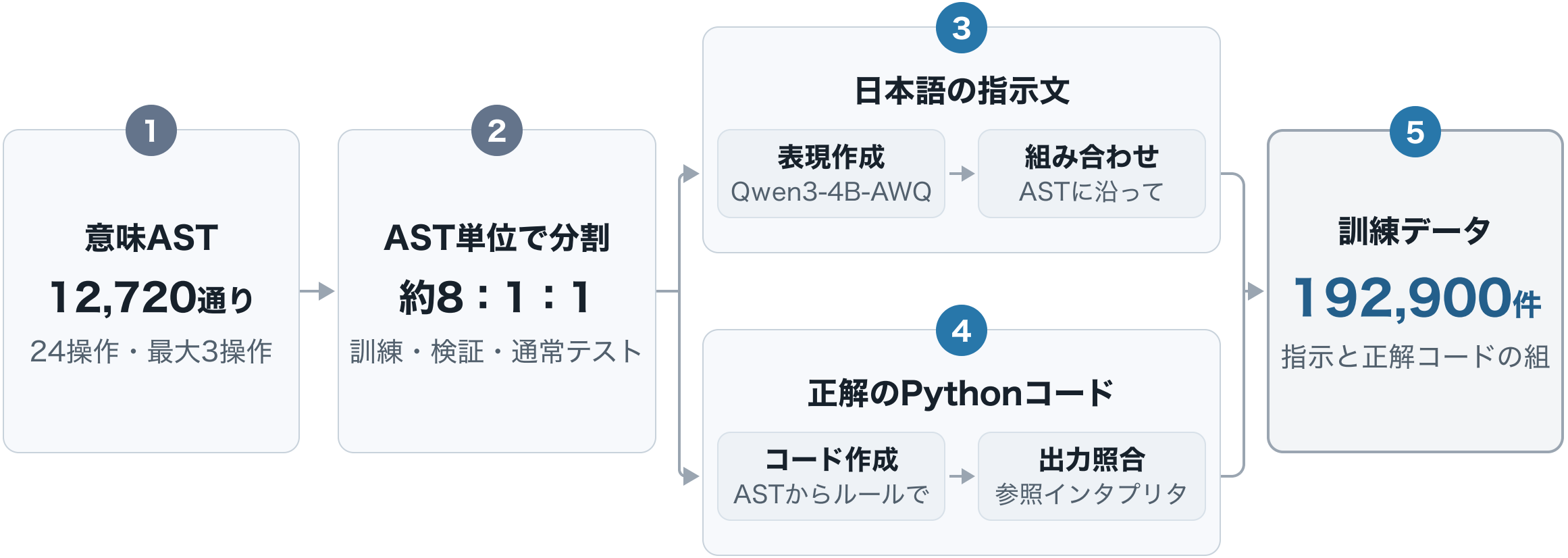
③ 異なる4操作
Bokuは7〜43%
訓練範囲の3操作を超えると低下

発表の流れ

	章	内容
①	**全体フロー・意味AST**	合成データ生成の流れと24操作の共通定義
②	**合成データ**	ASTの分割・日本語の言い換え・コード生成・検証
③	**学習**	トークナイザ・参照範囲・コード部分の損失・モデル構成と学習結果
④	**推論**	自己回帰・温度・次トークンの選択とコード生成
⑤	**評価**	ベンチマークの設計と結果・5モデルの失敗例

合成データ生成の全体フロー

同じ**意味AST**から日本語の指示文と正解コードを別々に作って組にする



意味AST（DSL）：24操作から最大3操作の組み合わせ

基本操作

種類	操作数	例
条件に合う数の抽出	10	偶数・奇数・正の値・kより大きい値
要素ごとの値の変換	8	kを加える・2倍・絶対値・二乗
順序の変更	3	昇順・降順・現在の順序の反転
一部の切り出し	3	先頭k個・末尾k個・1個おき

操作の組み合わせ

操作数	数え方	意味の種類
1操作	24	24
2操作	24 × 23	552
3操作	24 × 23 × 22	12,144
合計	24 + 552 + 12,144	**12,720**

意味ASTの例

日本語：奇数だけを残し、各要素を2倍して降順に並べる

```
{
    "sequence": [
        {"filter": ["odd"]},
        {"map": ["mul_const", 2]},
        {"order": "descending"}
    ]
}
```

意味AST単位での訓練・検証・テストの分割

意味ASTの分割　→　**各集合の中で**言い換え・コード変換

訓練 **約80%** ／ 検証 **約10%** ／ 通常テスト **約10%**

テスト	変更点・確認対象
通常	同じ基本操作の範囲で訓練に含まれない意味ASTの評価
言い換え	既知の単独操作への訓練で使わなかった日本語表現の適用
組合せ汎化	例：別々に学んだ「偶数抽出」と「降順整列」の組み合わせ
同じ操作の反復	同じ操作の2回・3回の連続適用（例：「各要素にkを加える」の2回適用）
境界値	空リスト・要素1個・条件を満たす要素なし・全要素同値・負数など

生成後の分割では同じ問題の表記違いが訓練とテストに混入
同じ意味ASTから作った言い換え・等価コードは同じ分割先に集約

Qwen3-4B-AWQによる日本語表現の生成と組み合わせ

表現辞書の作成と**承認済み表現の組み合わせ**による二段階の指示文生成

① 表現辞書の作成：Qwen3-4B-AWQ

各操作30候補の生成と人による意味の確認・承認
各操作20件以上の承認が目標

保存済みプロンプト（要約）

> 意味ASTと厳守事項に従い、指定件数の表現を作る。
> 操作・比較境界・入力順序を変えず、別の操作を加えない。
> 終止形と接続形を同じ意味で作り、指定のJSONだけを返す。

例：「偶数を残す」の生成候補

「偶数だけを残す」／「偶数だけを残し」
「偶数を選び出す」／「偶数を選び出して」

人による意味確認と承認済み表現のみの辞書登録

② 指示文の作成：ルールベース

意味ASTに沿った承認済み表現の組み合わせ

例：偶数を残す → 各要素にkを加える

選択する表現

「偶数の要素を取り出す」
「各要素にkを加える」

接続のルール

操作1し、操作2する

生成した日本語指示

「偶数の要素を取り出し、各要素にkを加える」

ルールベースで等価なPythonコード生成

同じ意味ASTでも異なる書き方のコード生成

内包表記による一括抽出

```python
def solve(xs, k):
    # 条件に合う要素のリストを返す
    return [
        value
        for value in xs
        if value % 2 == 0]
```

forループによる逐次抽出

```python
def solve(xs, k):
    result = []
    for value in xs:
        if value % 2 == 0:
            result = result + [value]
    return result
```

共通の入力：`[3, 2, 4, 2]`　→　どちらの出力も `[2, 4, 2]`

変更箇所	変換例
処理の構造	内包表記 ↔ forループ ／ 直接return ↔ 一時変数を使う複数行
名前・補足	変数名の変更 ／ コメント・型注釈の有無
条件・並べ替え	独立した条件の順序変更 ／ `reverse=True` ↔ 昇順ソート後の逆順（整数）

生成コードは完全重複なし／意味ASTの参照実装との結果一致の確認

学習用の合成データ192,900件

1件は「日本語の指示」と「正解コード」の組 ／ 意味ASTごとに最大20組

処理の長さ別の件数

処理の長さ	件数
1操作	460
2操作	8,680
3操作	183,760
合計	192,900

約95%が3操作

9,640種類は各20組
残る6種類は計100組

言い換えが20件未満の6種類

操作	訓練用の言い換え
ゼロだけを残す	11件
kの倍数だけを残す	16件
昇順に並べる	17件
負の値だけを残す	18件
各要素の符号を反転	19件
正の値だけを残す	19件

各操作1件の評価用表現を除外
訓練用の表現数に合わせた採用

BPEトークナイザ：頻出する文字列のトークン化

バイト単位から開始し語彙が **2,048** になるまで**最も多く隣り合う組の結合**の反復

<|bos|> <|task|> ⏎ 整数リストxsから 偶数だけを残す solve関数を書 いてください。 ⏎
<|code|> ⏎ def␣solve (xs) :⏎␣␣␣␣ return␣[v ␣ for␣ v ␣ in␣ xs␣if␣
v ␣ %␣2 ␣ ==␣0] <|eos|>

この1件は **29トークン**

「偶数だけを残す」が1トークンになるのは訓練データに頻出するため

設定	値	意味
語彙数	2,048	特殊トークン7個を含む結合を繰り返す際の語彙数の上限
最小頻度 `min_frequency`	5	訓練データに5回以上現れる組だけの結合
最大piece長 `max_token_length`	24	1トークンは最大24バイト（日本語なら8文字まで）

Decoder-onlyが自己回帰に向いている理由

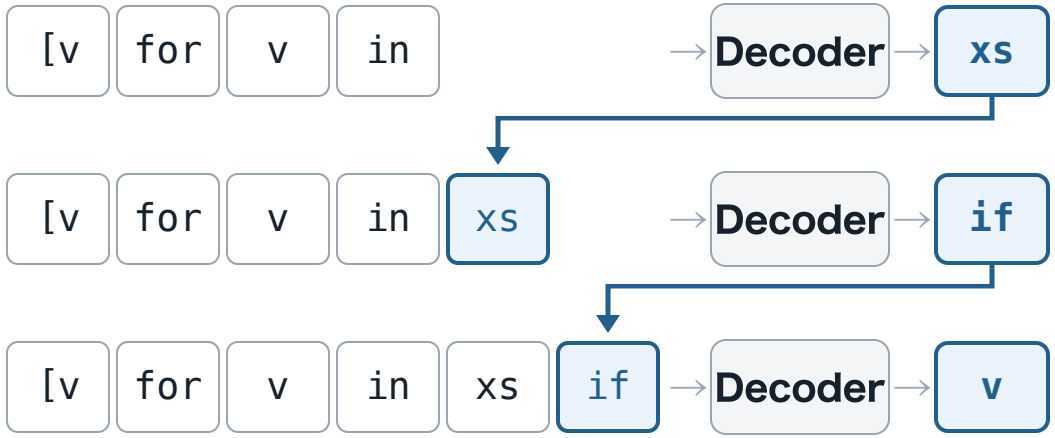
学習・生成に共通の**「今の位置までのトークンだけを見る」**というルール（例は `[v for v in xs if` の部分）

① 学習時：未来のマスク　○ 参照可能　× 参照不可

	[v	for	v	in	xs		当てるトークン
[v	○	×	×	×	×	→	for
for	○	○	×	×	×	→	v
v	○	○	○	×	×	→	in
in	○	○	○	○	×	→	xs
xs	○	○	○	○	○	→	if

→ 1つの系列で**5問ぶんの「次のトークン当て」**の同時学習

② 生成時：末尾への出力追加と予測の反復



学習時と生成時に共通の参照範囲による一貫した次トークン予測

学習時のコード：未来の位置への参照制限

```
# ① 各位置の表現からQ・K・Vを作る
query, key, value = self.qkv_proj(
    hidden_states
).chunk(3, dim=-1)
# …（各テンソルをheadごとに並べ替える）

# ② Q・Kに位置情報を加える
query, key = self.rope(query, key)

# ③ 現在位置までのK・Vだけを参照する
attended = F.scaled_dot_product_attention(
    query,
    key,
    value,
    dropout_p=self.dropout if self.training else 0.0,
    is_causal=True,
)
```

例：日本語指示から「偶数抽出」のコードへ

```
<|bos|><|task|>
整数リストxsから偶数だけを残すsolve関数を書いてください。
<|code|>
def solve(xs):
    return [v for v in xs if v % 2 == 0]
<|eos|>
```

return の末尾位置を基準とした区分	参照範囲
日本語指示・区切り	◯ 参照可能
`def solve(xs):` 〜 `return`	◯ 現在位置まで参照可能
後続の `[v for …]`	× 未来の位置は参照不可

正解コードの入力時も次に予測する答えは参照対象外

`is_causal=True` で未来へのAttentionの重みを0に固定
し指示とそこまでのコードから続きの予測

一つ先の正解を使った次トークンの学習

```
# ① 最後以外の位置の予測を取り出す
shifted_logits = logits[:, :-1, :].contiguous()

# ② 正解を一つ先の位置にずらす
shifted_labels = labels[:, 1:].contiguous()

# ③ 正解トークンの確率から損失を計算
loss = F.cross_entropy(
    shifted_logits.view(
        -1, shifted_logits.size(-1)),
    shifted_labels.view(-1),
    ignore_index=-100,
)
```

今の位置の出力と「次の正解」の比較

出力を使う位置	予測すべき正解
コード直前	最初のコードtoken
最初のコードtoken	2番目のコードtoken
最後のコードtoken	EOS（生成終了）

$$\mathcal{L} = -\frac{1}{|C|} \sum_{t \in C} \log P_\theta(x_t \mid x_{<t})$$

C：コード開始後〜EOSの位置（指示・paddingは除外）

訓練時は**正解の続きの予測**
生成時は**自分が出したトークンの続きの予測**

日本語指示は条件として読みコード部分だけで損失計算

学習時は**指示と正解コードを両方入力**し次の正解トークン予測

入力系列	**日本語の指示**
	偶数だけを残すsolve関数を書いてください
		<|code|>	**正解のPythonコード**
			`def solve(xs):`
			`return [v for v in xs if v % 2 == 0]`
				EOS

正解ラベル	−100	−100	各コードトークンのID	EOSのID
損失の対象	損失計算の対象外	対象外	**損失の計算対象**	**計算対象**

日本語の指示を参照したコードの続きの予測　→

```
labels[:] = IGNORE_INDEX  # -100
labels[target_start:] = token_ids[target_start:]
# target_start は <|code|> の直後
```

入力範囲と損失の計算対象は別

指示文の予測は採点対象外

コード側の損失により

指示を処理する重みも学習

モデル構成とトークナイザー

項目	1M	5M	15M
総パラメータ数	**1,016,704**	**5,065,472**	**15,735,168**
隠れ次元 `d_model`	128	256	384
層数 `n_layers`	3	5	8
Attentionのhead数	4	4	6
1 headの次元	32	64	64
FFNの中間次元 `d_ff`	256	704	1,024
この後に示す学習曲線	3 epoch	1 epoch	1 epoch

共通のトークナイザー

BPE ／ 語彙数 2,048

最小頻度 5 ／ 最大piece長 24

最大系列長 256 token

埋め込み・出力層の重み共有なし

共通の学習設定

AdamW ／ β = (0.9, 0.95) ／ batch size 512

重み減衰 0.1（行列重み）／ dropout 0

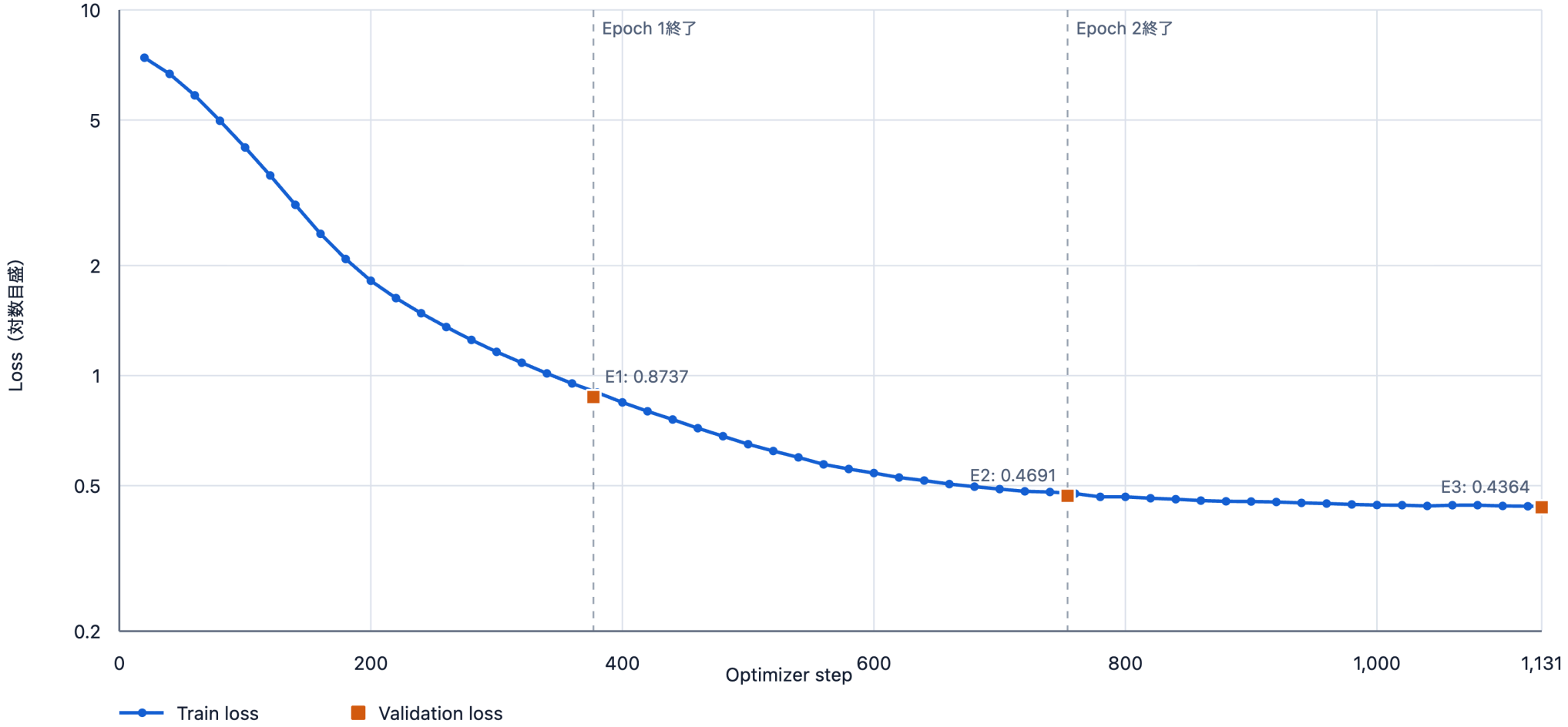
5%の線形warmup → cosine減衰

学習率 $3\times10^{-4} \rightarrow 3\times10^{-5}$

勾配クリッピング 1.0 ／ bfloat16での計算

1M：3 epochまでの損失の推移

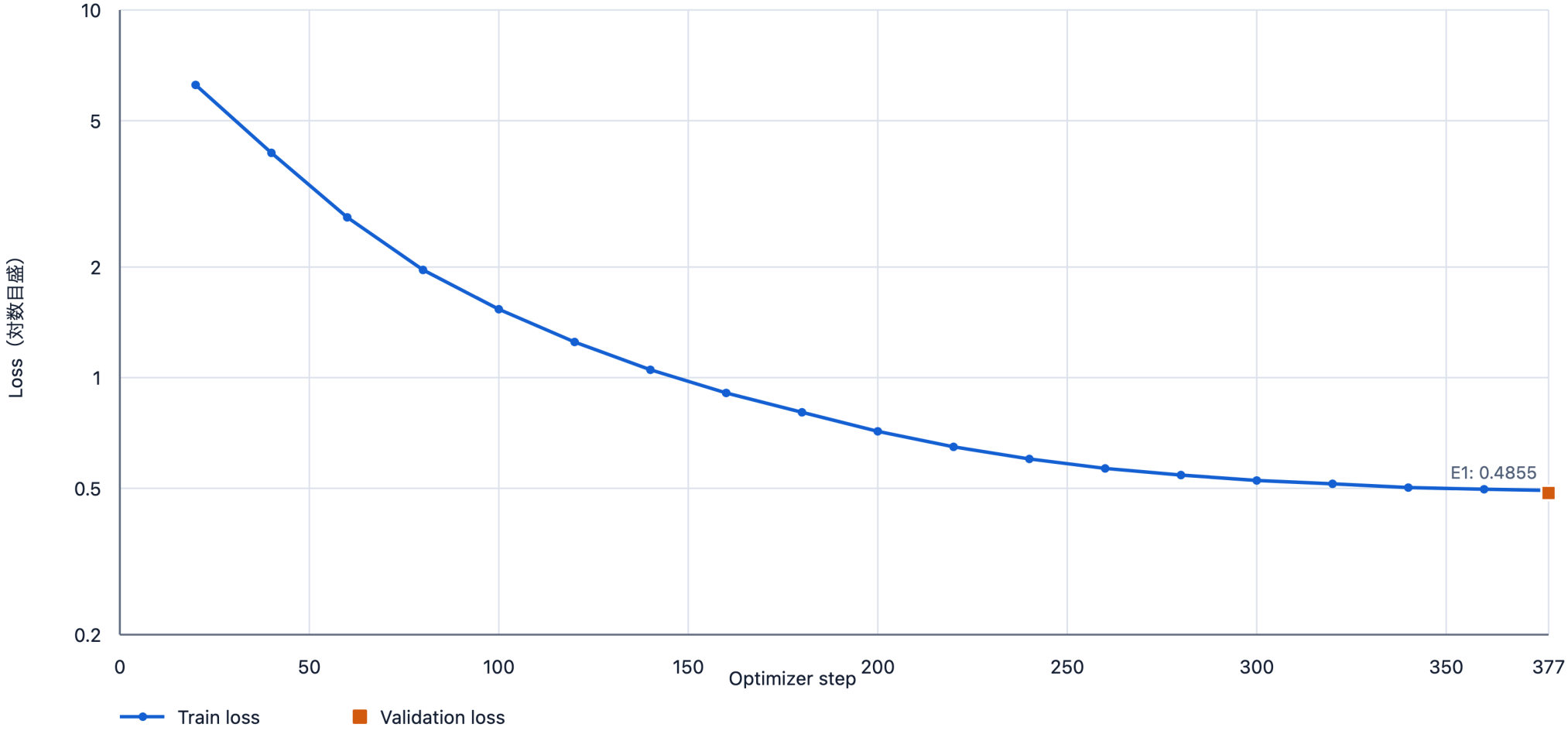
3 epochの学習（訓練損失は20 stepごと／検証損失は各epoch終了時の実測値）



val loss：1 epoch 0.8737 ／ 2 epoch 0.4691 ／ 3 epoch 0.4364

5M：損失の推移

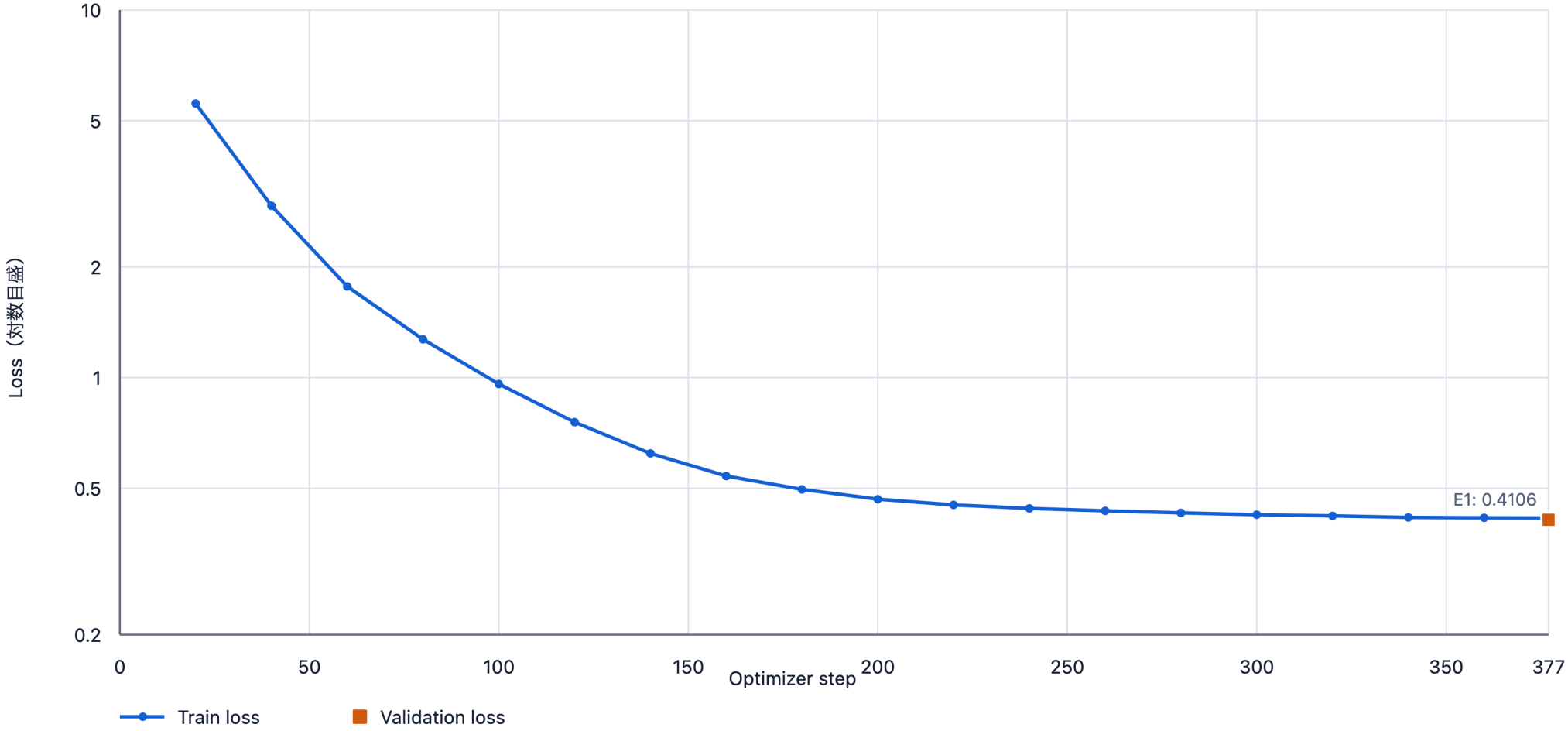
1 epochの学習における訓練損失と検証データでの損失



学習終了時の val loss：0.4855

15M：損失の推移

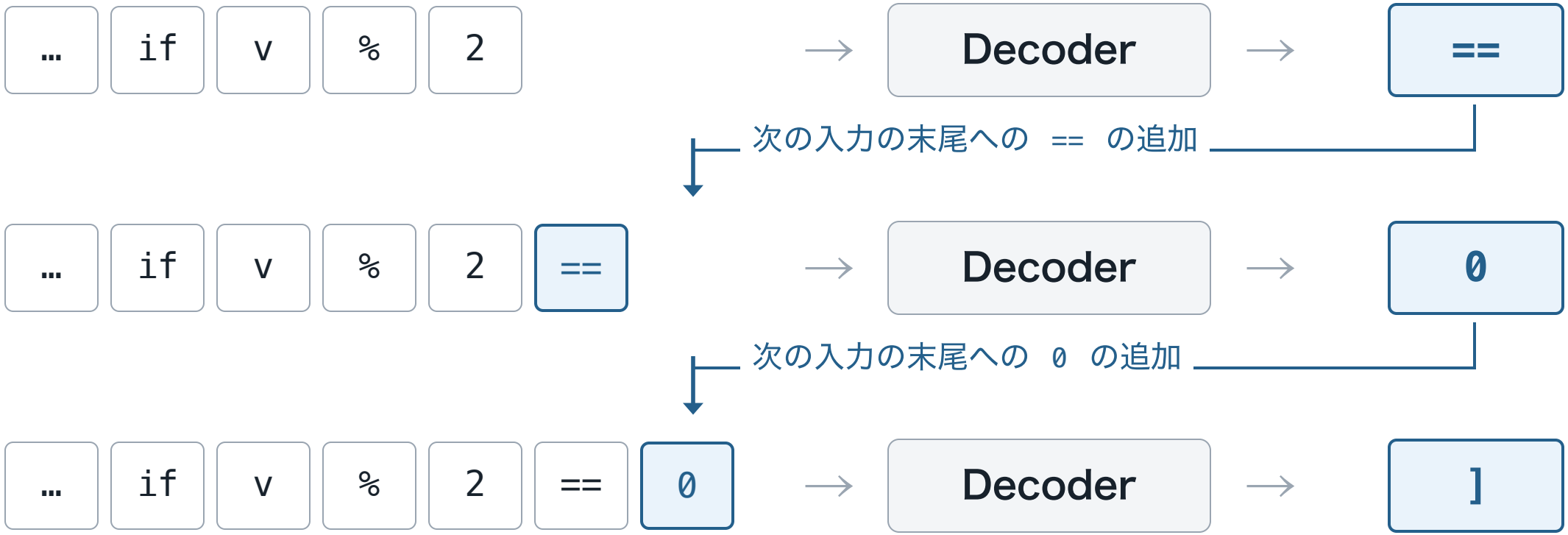
1 epochの学習における訓練損失と検証データでの損失



学習終了時の val loss：0.4106

自己回帰：出力の追加と次トークン予測の反復

訓練データ1件は特殊トークンで区切った「指示＋コード」の1本の系列
直前までのトークンから次のトークン予測



学習時：系列の途中までを入力し実際に続くトークンの予測
生成時：`<|eos|>` が出るまで予測の反復

温度：次トークン候補の確率分布の調整

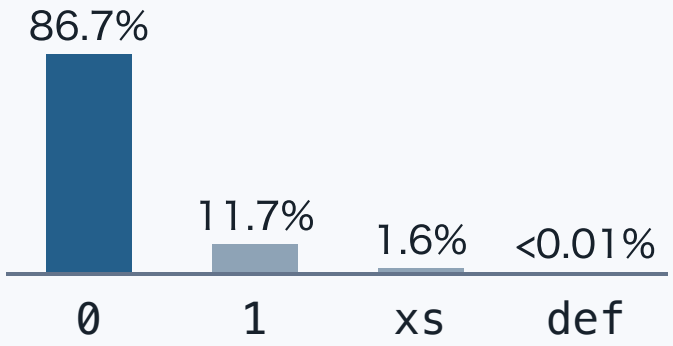
計算例 `… if v % 2 == ?`

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}} \quad T > 0$$

固定するスコア（logit）：`0 → 3` ／ `1 → 2` ／ `xs → 1` ／ `def → −2`

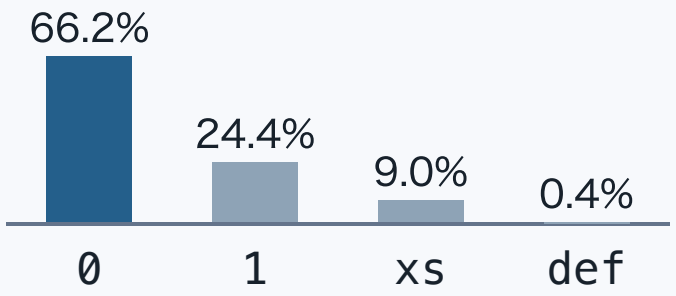
T = 0.5

高確率候補への集中



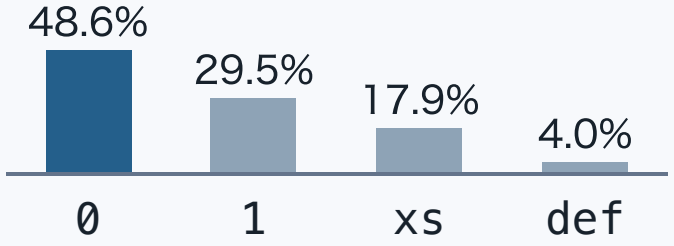
T = 1

基準の分布



T = 2

他の候補への確率の分配



確率に従う抽選：高い温度による候補の多様化

最大確率の選択：正の温度変更でも選択不変（今回のBoku評価）

次のトークン候補への確率の割り当て

`… if v % 2 ==` の続きも候補のlogitを確率に変換し1つの候補の選択

`… v % 2 == ?` → **transformer** → 次のトークン候補に対するlogit

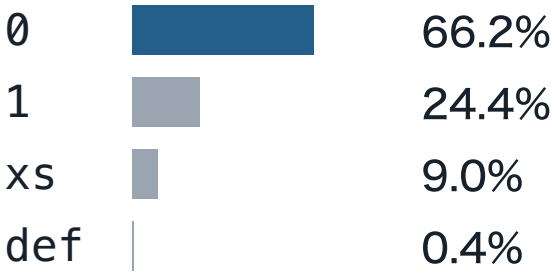
① logit

logit：負の値も取る確率変換前のスコア



softmax →

② 合計1となる確率



選択 →

③ 最も確率が高い候補の選択

`0  66.2%`

↓ 入力の末尾に追加

`… v % 2 == 0`

> logitを**softmax**で確率に変換し最も確率の高い候補を次のトークンとして採用

生成時のコード：最後の予測の選択と入力への追加

```
# …（生成ループ内）
# ① 最後の位置の候補の点数を得る
next_logits = model(input_ids).logits[
    :, -1, :
]

# ② 最も確率の高い候補のIDを選ぶ
next_ids = torch.argmax(
    next_logits, dim=-1
)
# …（EOS判定・出力保存）
appended = torch.tensor(
    append_values,
    dtype=torch.long, device=device
)
# ③ 次回の入力の末尾に追加する
input_ids = torch.cat(
    (input_ids, appended[:, None]), dim=1
)
```

「偶数抽出」のコードの続きの生成

```
<|bos|><|task|>
整数リストxsから偶数だけを残すsolve関数を書いてください。
<|code|>
def solve(xs):
    return [
```

処理	上の入力から1トークン進む例
① 最後の位置の使用	入力末尾から続きの候補の予測
② 候補の選択	例：`v` のトークンIDの選択
③ 末尾に追加	`return [` → `return [v`
次のループ	日本語指示と追加済みコード全体の入力

正解の続きを渡す学習時と異なり生成時は自分が選んだトークンの追加

`[:, -1, :]` は「全系列・最後の位置・全候補」
`<|eos|>` で終了 ／ それまでは上限まで予測の反復

モデル規模・学習回数による違い：五つのテスト別結果

各モデルの列は **合格数・pass@1（合格率）**
各問1本を生成し検査入力をすべて満たせば合格

テスト	1M・1 epoch	1M・3 epoch	5M・1 epoch	15M・1 epoch
通常（24,040問）	446問（1.86%）	23,950問（99.63%）	23,261問（96.76%）	**24,003問（99.85%）**
組合せ汎化（13,400問）	103問（0.77%）	**13,311問（99.34%）**	12,657問（94.46%）	13,188問（98.42%）
日本語の言い換え（30問）	2問（6.67%）	**15問（50.00%）**	8問（26.67%）	**15問（50.00%）**
同じ操作の反復（23,040問）	508問（2.20%）	22,945問（99.59%）	22,370問（97.09%）	**22,971問（99.70%）**
境界値（24,040問）	408問（1.70%）	23,950問（99.63%）	23,261問（96.76%）	**24,003問（99.85%）**

1Mは3 epochで大きく改善
一方言い換えは1M・3 epochと15M・1 epochでも50%

ベンチマークの役割と自分の研究での作り方

モデルの能力と限界を観測し次に取り組む研究課題の明確化

成功例だけで判断せず共通の基準で能力向上の定量的な確認

設計・評価の手順	自分の研究での検討点
① 測定する能力の選定	確認したい能力とその変化
② 課題と正解基準の作成	変更する条件・固定する条件・正誤の判定方法
③ 共通条件での測定・分析	成功・失敗の条件の分析と比較条件・作成手順の記録
④ 次の課題への接続	残る誤りの分析と高得点で差が見えない場合の評価の見直し

一例：組み合わせ汎化の評価

学習済みの操作を新しい組み合わせでも実行する能力の評価

今回は既知の日本語表現を用いて構成を変更し生成コードの実行結果で正誤判定

データセットの構成：9種類・872問

操作数	問題の種類	問数	構成
1	複数操作で学んだ表現の1操作への適用	96	24操作 × 4表現
2	初めて組み合わせる2操作	100	5ペア × 両順序 × 10表現
2	同じ操作の2回反復	96	24操作 × 4表現
2	3操作で学んだ順序の2操作への適用	88	88種類の操作列
3	初めてのペアを含む3操作	100	100種類の操作列
3	同じ操作の3回反復	96	24操作 × 4表現
4	同じ操作の4回反復	96	24操作 × 4表現
4	異なる4操作の組み合わせ	100	100種類の操作列
4	2操作の組の2回反復	100	A・B・A・Bの100種類

各問題で1つのコード生成
正答率は**179通りの共通テスト入力ですべて合格した問題の割合**

Astraの評価設定：思考low・ツール使用禁止

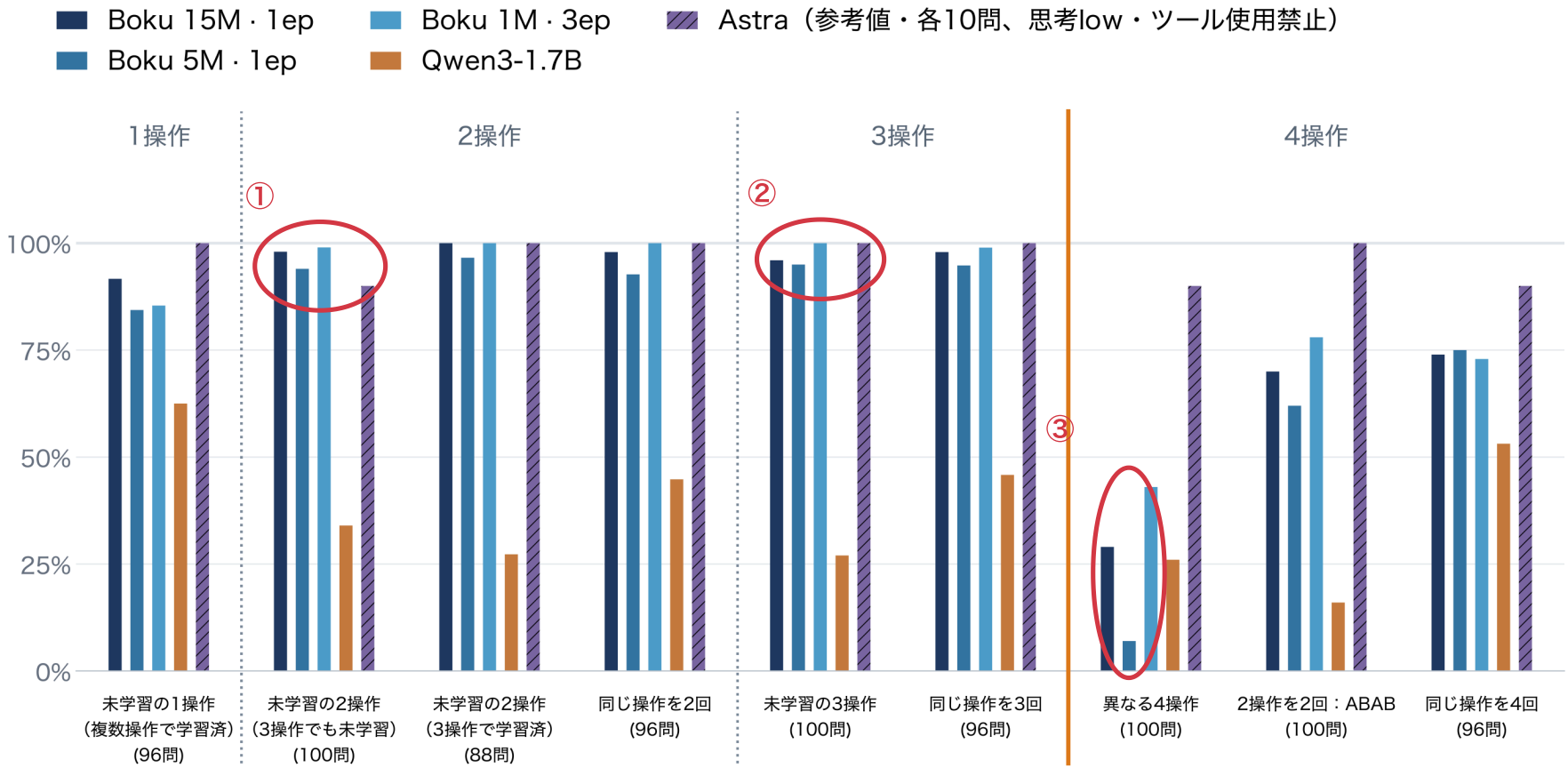
項目	設定
モデル・推論設定	`gpt-6-astra` ／ reasoning effort：**low**
評価対象	872問から固定seedで抽出した各カテゴリ10問・計90問
会話・回答	1問ごとに独立チャット ／ 最初の回答1回の採点 ／ 修正・再生成なし
ツール	使用禁止の指示 ／ **90問すべての実行記録で使用0件の確認**

各問題の冒頭に与えた指示：

> ツールは一切使用せず、このメッセージだけから回答してください。ファイル閲覧、検索、コード実行、他のエージェントへの依頼は行わないでください。回答は一度だけ、指定されたコードのみを出力してください。

ツールの技術的な無効化ではなく**指示による禁止と記録確認**
採点は回答保存後 ／ Astraへの結果のフィードバックなし

評価結果の注目点3か所



① 未学習の2操作
1M・3 epoch：99%
Astra参考値：90%

② 未学習の3操作
1M・3 epoch：100%
学習済みの操作による新しい組み合わせ

③ 異なる4操作
Bokuは7〜43%
訓練範囲の3操作を超えると低下

ベンチマークの失敗例一覧：5モデルの誤りと実行結果

モデル	確認された誤り	正解	実際の出力
① Astra	参照定義の降順に対する昇順整列	[4, 2]	[2, 4]
② Qwen3-1.7B	3倍に対するk倍への置換	[3]	[1]
③ Boku 1M・3 epoch	最後の「先頭k個」の欠落	[100]	[100, 100]
④ Boku 5M・1 epoch	降順整列の欠落と切り出しの先行	[0]	[]
⑤ Boku 15M・1 epoch	降順整列の欠落	[66]	[28]

Bokuの異なる4操作・100問の集計：合格／出力不一致は1M：43／57・5M：7／93・15M：29／71
3条件とも構文・実行エラーの判定は0件

失敗例① Astra：参照定義と整列方向の不一致

> 整数リストxsから偶数を切りとって、値を大小順に並べるsolve関数を書いてください。

参照ASTの処理：**偶数抽出 → 降順整列**

実際に生成したコード

```
def solve(xs: list[int], k: int) -> list[int]:
    return sorted([x for x in xs if x % 2 == 0])
```

コード内の処理

偶数抽出後の `[4, 2]`

↓ `sorted` の既定の昇順

出力 `[2, 4]`

実行して分かった誤り

偶数抽出後に昇順で整列
参照定義の降順との不一致

検査入力

```
xs = [5, 4, 3, 2, 1]
k = 5
```

正解	実際の出力
[4, 2]	**[2, 4]**

失敗例② Qwen：3倍からk倍への置き換え

> 整数リストxsからすべてを三倍するsolve関数を書いてください。

参照ASTの処理：**各要素の3倍**

実際に生成したコード

```
def solve(xs, k):
    result = list(xs)
    for i in range(len(result)):
        result[i] *= k
    return result
```

コード内の処理

乗数に固定値3ではなく引数 `k`

`k = 1` の検査入力では1倍

実行して分かった誤り

指定された3倍ではなく
引数kによる乗算

検査入力

```
xs = [1]
k = 1
```

正解

[3]

実際の出力

[1]

失敗例③ Boku 1M・3 epoch：最後の切り出しの欠落

> 整数リストxsと整数kを受け取り、絶対値を導き出して、偶数だけを切り出して、大きい順に並べて、頭k個を抜き出すsolve関数を書いてください。

参照ASTの処理：**絶対値 → 偶数抽出 → 降順整列 → 先頭k個**

実際に生成したコード

```python
def solve(xs: list[int], k: int) -> list[int]:
    result: list[int] = []
    for x in xs:
        result += [abs(x)]
    output: list[int] = []
    for x in result:
        if x % 2 == 0:
            output += [x]
    output = sorted(output, reverse=True)
    return output
```

実行して分かった誤り

最後の「先頭k個」の
切り出しの欠落

検査入力

```
xs = [-100, 100]
k = 1
```

正解

[100]

実際の出力

[100, 100]

失敗例④ Boku 5M：操作の欠落と実行順序の変化

> 整数リストxsと整数kを受け取り、絶対値を導き出して、偶数だけを切り出して、大きい順に並べて、頭k個を抜き出すsolve関数を書いてください。

参照ASTの処理：**絶対値 → 偶数抽出 → 降順整列 → 先頭k個**

実際に生成したコード

```
def solve(xs: list[int], k: int) -> list[int]:
    result: list[int] = []
    for x in xs:
        result += [abs(x)]
    output: list[int] = result[0:k]
    result = []
    for value in output:
        if value % 2 == 0:
            result += [value]
    return result
```

実行して分かった誤り

降順整列の欠落
偶数抽出より前での
「先頭k個」の切り出し

検査入力

```
xs = [-5, -1, 0, 1, 5]
k = 2
```

正解

[0]

実際の出力

[]

失敗例⑤ Boku 15M：降順整列の欠落

> 整数リストxsと整数kを受け取り、絶対値を導き出して、偶数だけを切り出して、大きい順に並べて、頭k個を抜き出すsolve関数を書いてください。

参照ASTの処理：**絶対値 → 偶数抽出 → 降順整列 → 先頭k個**

実際に生成したコード

```python
def solve(xs: list[int], k: int) -> list[int]:
    # 各要素の絶対値を取る
    result: list[int] = []
    for value in xs:
        result += [abs(value)]
    # 偶数だけを残す
    output: list[int] = [x for x in result if x % 2 == 0]
    # 先頭からk個を取る
    output = output[0:k]
    return output
```

実行して分かった誤り

降順整列の欠落
元の順序での「先頭k個」の出力

検査入力

```
xs = [53, -28, 35, -66, -69, -14, -23]
k = 1
```

正解

[66]

実際の出力

[28]

考察

できたことと残った誤りを次の学習につなげる

今回の結果の意義と次の研究課題

指示側の予測を損失に含めず
コード側だけを採点する学習によるコード生成の実現

今回の課題における**組み合わせ汎化**の確認：学習済み操作の新しい構成での利用
1M・3 epoch：未学習の2操作 **99/100** ／ 未学習ペアを含む3操作 **100/100**

今後の課題：比較条件を揃えたより大きなモデルとの追加比較と
評価設計・作成手順の明文化

- 比較条件：訓練回数・プロンプト・生成上限などの記録と揃えられる条件の統一
- 評価設計：既知・未知の範囲と検出対象とする誤りの定義
- 作成手順：問題選定・訓練との照合・採点方法・再現手順の公開に向けた整理

考察：問題単位の合否を次の訓練へ生かす方針

既知の表現でも4操作の組み合わせで残る課題

1M・3 epoch：未学習の2操作 **99 / 100** ／ 未学習ペアを含む3操作 **100 / 100**
学習済みの表現を用いた新しい2・3操作の構成への対応

異なる4操作では **43 / 100**
観察された操作の欠落・順序の誤りと要求された処理を最後まで揃える課題

問題単位の採点を訓練へ反映する方法

pass@1：コードの長さによらず**1問1件としての採点**
各問題の検査入力をすべて満たすかによる評価

得点の比較に加え採点結果を訓練へ反映する方法の検討

2・3操作での達成と4操作での不足を踏まえた
訓練データの構成・学習方法の見直し

まとめ：二つの系統とその先にある学習方法

自然言語（物語）系

TinyStories・SimpleStories
語彙・文体を限定した合成物語で学習

↓

文章の良さに対する人・LLMの評価
LLM-as-a-Judge：LLMによる採点

↓

RLHFへの接続：
人の好みを表す報酬モデルを用いた学習

math・code系

phi-1・TinyGSM
対象ドメインの合成・選別データで学習

↓

テスト・実行による評価
正誤の0・1判定

↓

RLVRへの接続：
検証可能な報酬を用いた学習

Boku-nano：参照インタプリタによる0・1判定
合成データの選別とベンチマークに利用

本研究の成果：**制御した合成データ＋実行検証**による小型モデルの組合せ汎化

デモ：学習済みモデルのブラウザ実行

日本語入力 → Qwen3-0.6Bによる文型整形 → Boku-nano 15MによるPythonコード生成

公開サイト：yuuono.github.io/bokunano

Appendix：追加解析と補足資料

本編評価を補足する日本語表現への弱さとモデル内部の挙動の解析

内容	確かめること
A. 日本語表現への頑健性	言い換え・指示冒頭の変更による生成結果の変化
B. Attentionの可視化と介入	参照先と指示情報のコード生成への寄与
C. 追加解析のまとめと限界	結果から言える範囲と追加検証の必要な点
D. 全体図	データ生成・学習・評価・デモのつながり

高い全体精度と未学習の言い換えへの弱さ

1M・3 epochと15M・1 epochの比較 ／ 各セルは合格数（合格率）

テスト（評価件数）	1M・3 epoch	15M・1 epoch
通常（24,040問）	23,950（99.63%）	24,003（99.85%）
組合せ（13,400問）	13,311（99.34%）	13,188（98.42%）
日本語の言い換え（30問）	**15（50.00%）**	**15（50.00%）**
同じ操作の反復（23,040問）	22,945（99.59%）	22,971（99.70%）
境界値（24,040問）	23,950（99.63%）	24,003（99.85%）

全体の合格率はどちらも約99.6%
未学習の言い換えの合格率は50%

先行研究：日本語の書き方による正解率の変化

Gan and Mori（2023）：意味や指示の近い5種類のプロンプトの比較

プロンプト：モデルに渡す指示文や質問文

日本語の課題	モデル	正解率の範囲
2文の意味関係を分類するJNLI	GPT-4	**25.44〜49.21%**

意味の近い指示でも書き方によって性能が大きく変わる可能性

先行研究：言い換えによる回答傾向の変化

Takayama et al.（2026）：
日本語を含む「はい／いいえ」質問の評価

表現の変え方	日本語での観察
同意を求める「〜ですよね？」	道徳判断で「はい」が増える傾向
反対語への置き換え	答えの反転を考慮しても一貫性が低下

同じ判断を求める質問でも表現によって変わる回答傾向

モデル評価における複数の言い回しを含めた検証の必要性

指示の冒頭だけを変えた比較

条件	モデルに渡す指示
変更前	整数リストxsから偶数だけを残すsolve関数を書いてください。
変更後	整数リストxsに対して、偶数だけを残すsolve関数を書いてください。

後半の操作の指示をそろえ冒頭だけが異なる14操作の比較

同じモデル・同じ検査入力による追加学習なしでのコード検証

5Mモデル：合格数12件から1件への低下

5M・1 epoch：比較した14操作の合格数

指示の冒頭	合格数を表す図	合格数・合格率
整数リストxsから	■■■■■■■■■■■■■□□	12 / 14・85.7%
整数リストxsに対して、	■□□□□□□□□□□□□□	1 / 14・7.1%

■は合格1件・□は不合格1件（左から合格数を並べた図）

1M・3 epochでも同じ14操作で 12 / 14 → 9 / 14 に低下

冒頭の変更に伴う5Mモデルの合格数の大幅な低下

失敗例：偶数抽出から別の処理への変化

変更後の指示：**整数リストxsに対して、偶数だけを残すsolve関数を書いてください。**

	指示どおり	5M・1 epoch	1M・3 epoch
処理	偶数のみの抽出	順序反転後のk以上の抽出	正の数のみの抽出
同じ検査入力	`xs = [0], k = 1`	`xs = [0], k = 1`	`xs = [0], k = 1`
出力	`[0]`	`[]`	`[]`

両モデルとも正しい構文

指示された処理と生成された処理の意味の不一致

表現への頑健性に見られるモデル間の差

冒頭だけを変えた14操作の合格数と学習回数

モデル	「xsから」	「xsに対して、」
1M・3 epoch	12 / 14	9 / 14
5M・1 epoch	**12 / 14**	**1 / 14**
15M・1 epoch	12 / 14	12 / 14

1M・3 epochでは3件の減少／15M・1 epochでは合格数の維持

モデル・学習回数によって異なる変化の大きさ

訓練データの文型との結び付きの可能性

訓練データで「整数リストxsに対して、」と始まる指示は84件

その文型で学習した処理	件数
1操作	0
2操作	1
3操作	83

この文型を複数の操作と結び付けて学習した可能性

原因の確定に向けた合成データを変えた追加比較の必要性

モデルが情報を読み取る仕組み

アテンション：次のコード生成時に情報の参照先を決める仕組み

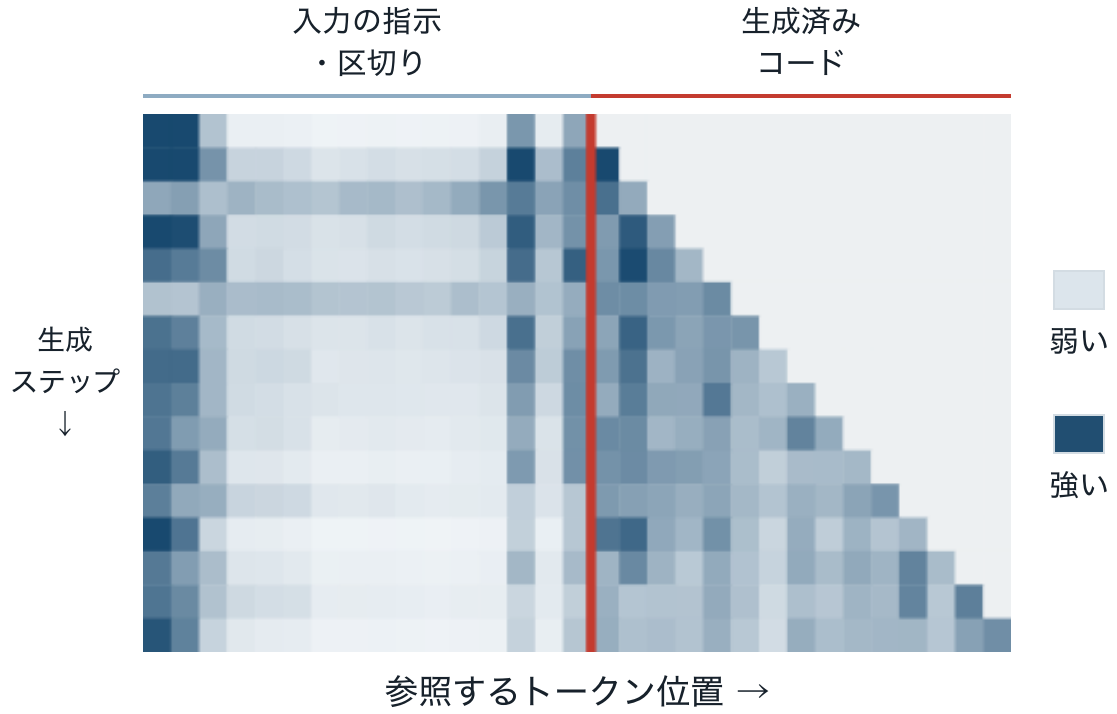
参照できる情報	内容
日本語の指示	「偶数」「残す」など求める処理
すでに生成したコード	変数名・構文・処理の続き
開始の印など	入力の始まりや区切り

参照割合だけでは確定できない各情報の必要性

情報の受け渡しを止めた場合の生成結果の変化の検証

Attentionの可視化：Q・Kによる参照の強さの復元

```
# ① forward hookで各層のQ・Kを取得
# …（capture_qkv内でRoPEも適用）
# ② 生成する位置のQと、過去のKを比較
score = torch.einsum(
    "hd,htd->ht", query[:, -1], key
) / math.sqrt(key.size(-1))
weights = torch.softmax(score, dim=-1)
attention.append(weights)
# …（生成stepごとにrecordsへ保存）
# ③ 層・head・stepごとの行を色に変換
row = record["attention"][layer, head]
# …（positionは存在する参照位置）
fill = blue(float(row[position]), maximum)
```

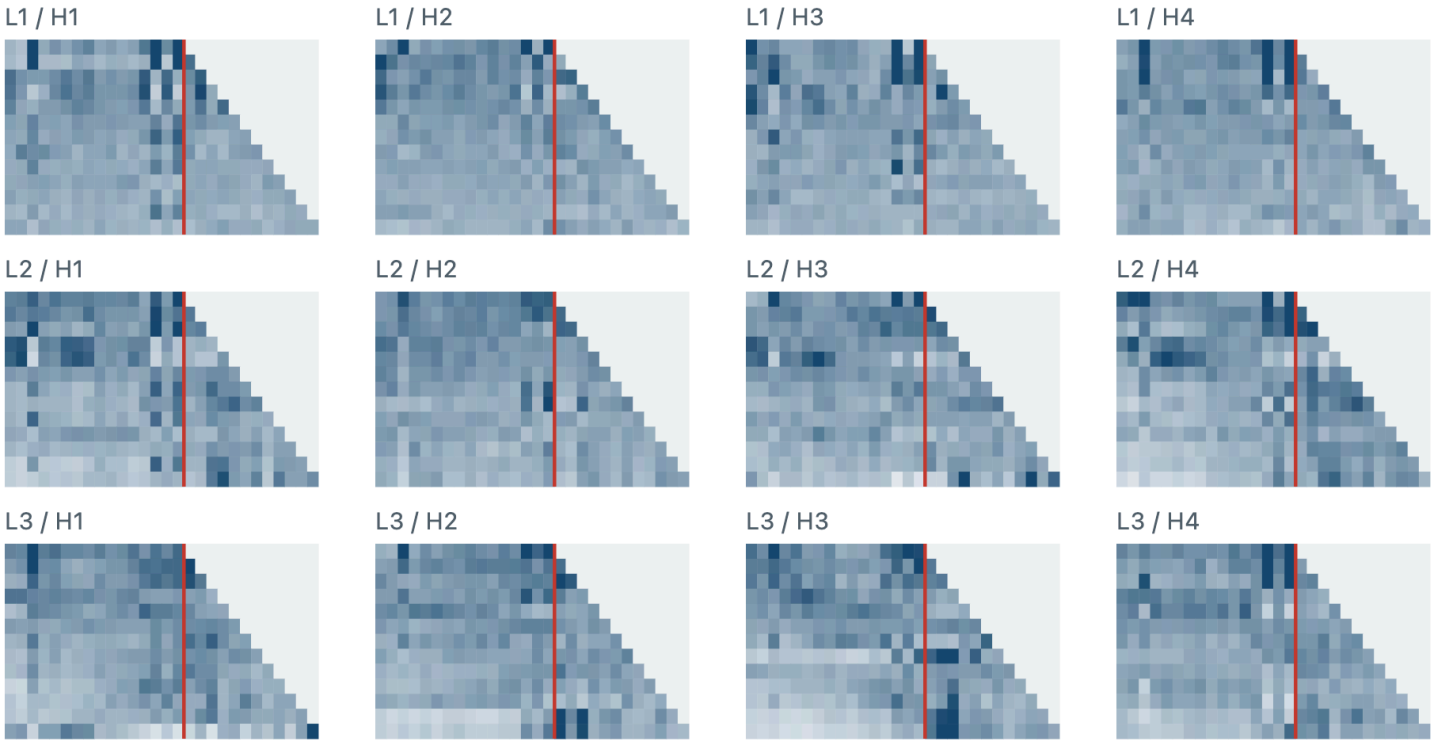


赤線：指示側と生成コード側の境界

灰色の領域：その生成時点ではまだ存在しない位置

1M・1 epoch：attention の可視化

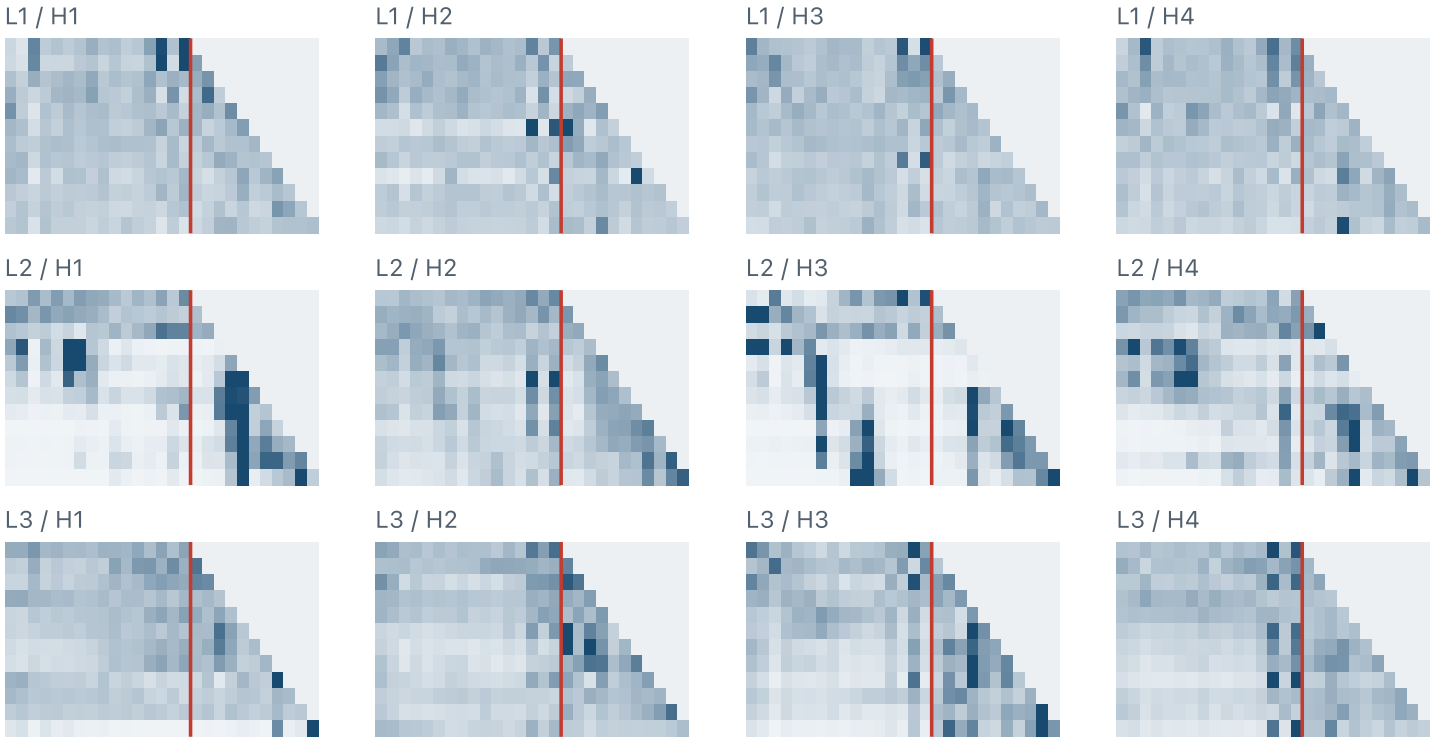
同じ指示：**各要素を絶対値にして、降順に並べる**



生成結果：未定義の result の参照による実行失敗

1M・3 epoch：attention の可視化

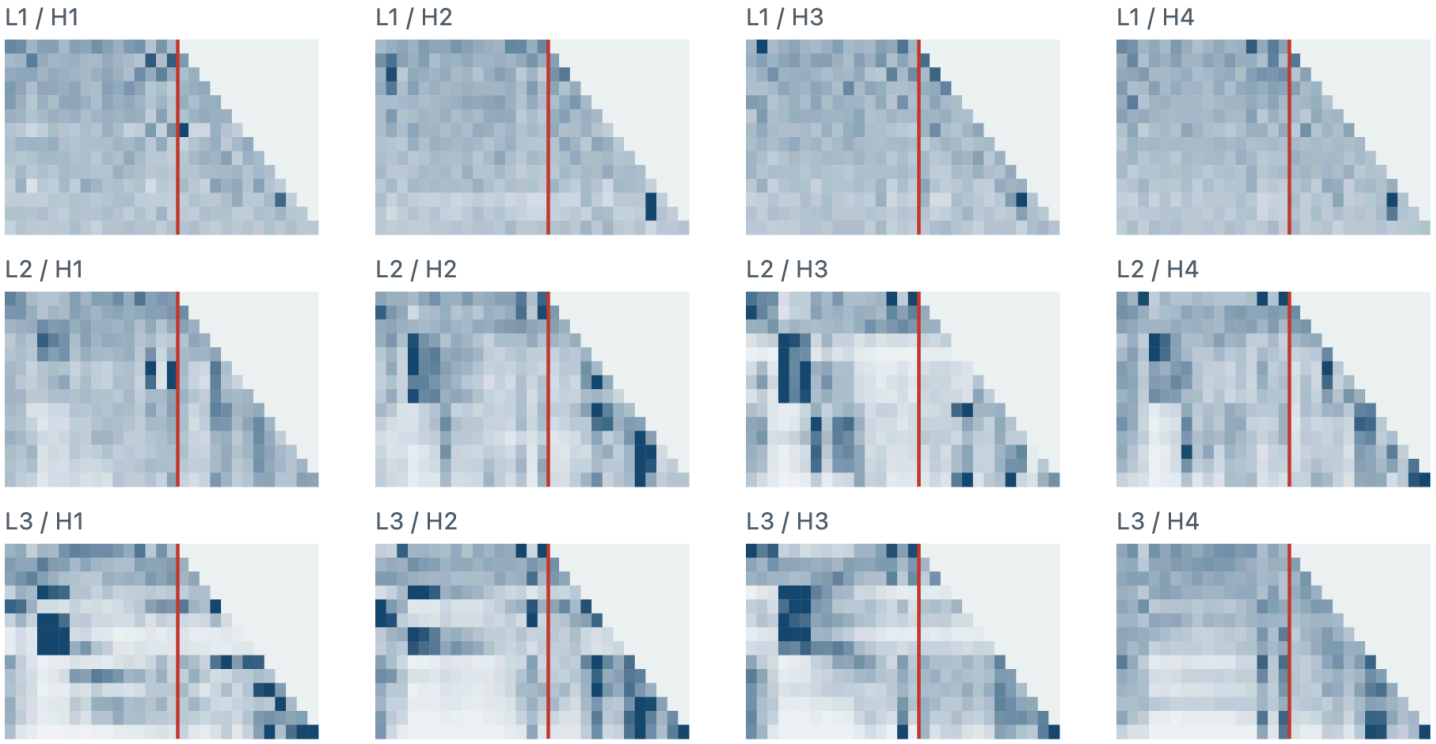
同じ指示：**各要素を絶対値にして、降順に並べる**



生成結果：未定義の output の参照による実行失敗

5M：attention の可視化（L1〜L3）

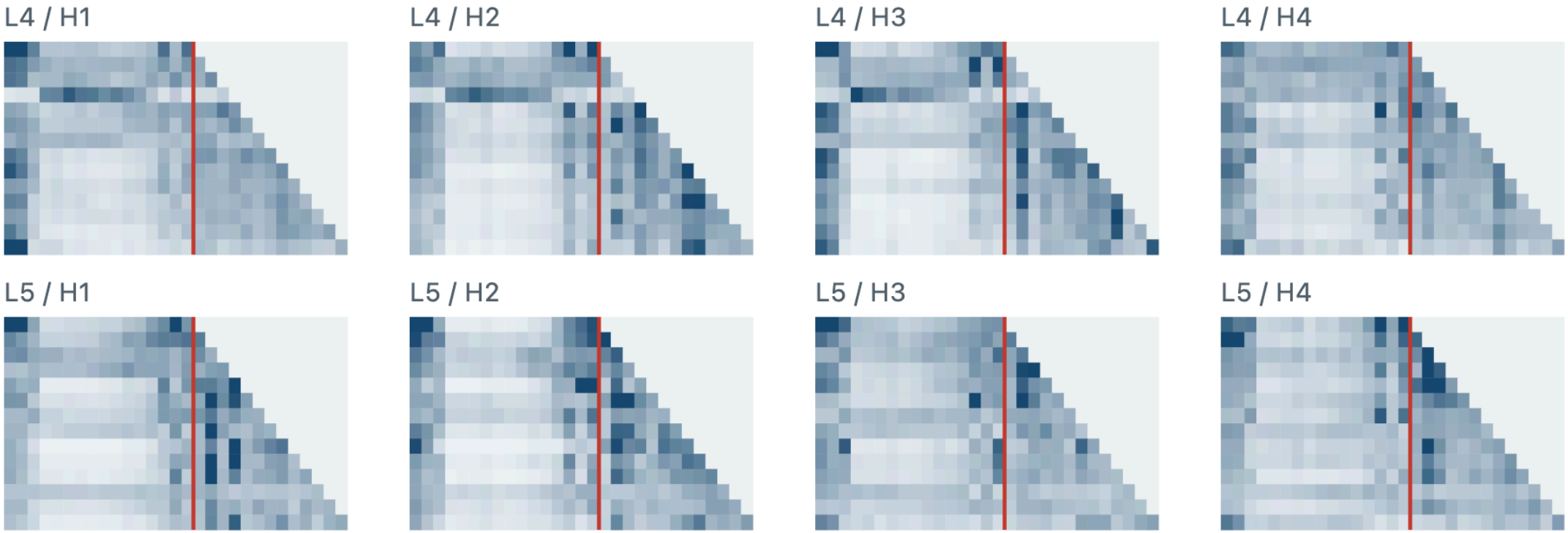
同じ指示：**各要素を絶対値にして、降順に並べる**



生成結果：絶対値処理に加えた不要なk加算と降順整列の欠落

5M：attention の可視化（L4〜L5）

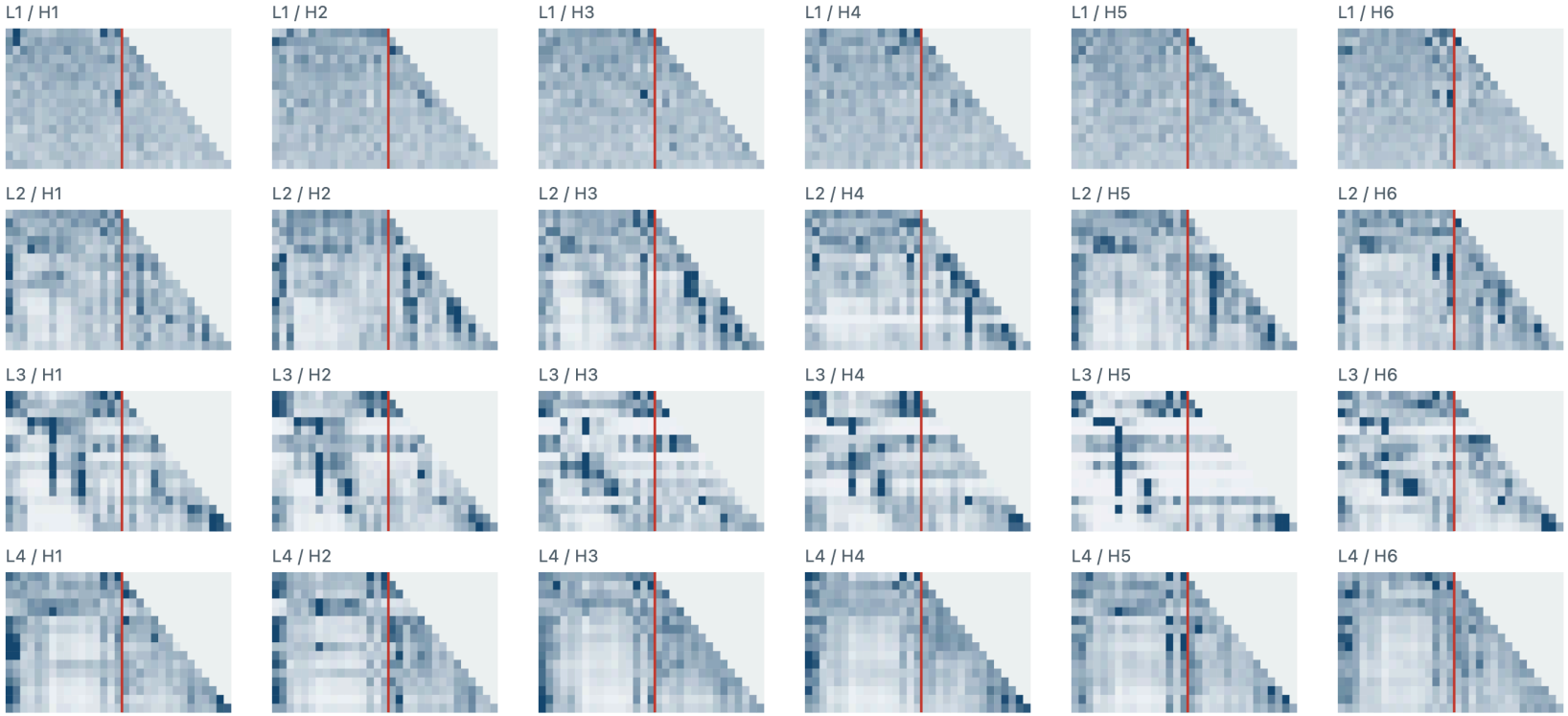
同じ指示：**各要素を絶対値にして、降順に並べる**



生成結果：絶対値処理に加えた不要なk加算と降順整列の欠落

15M：attention の可視化（L1〜L4）

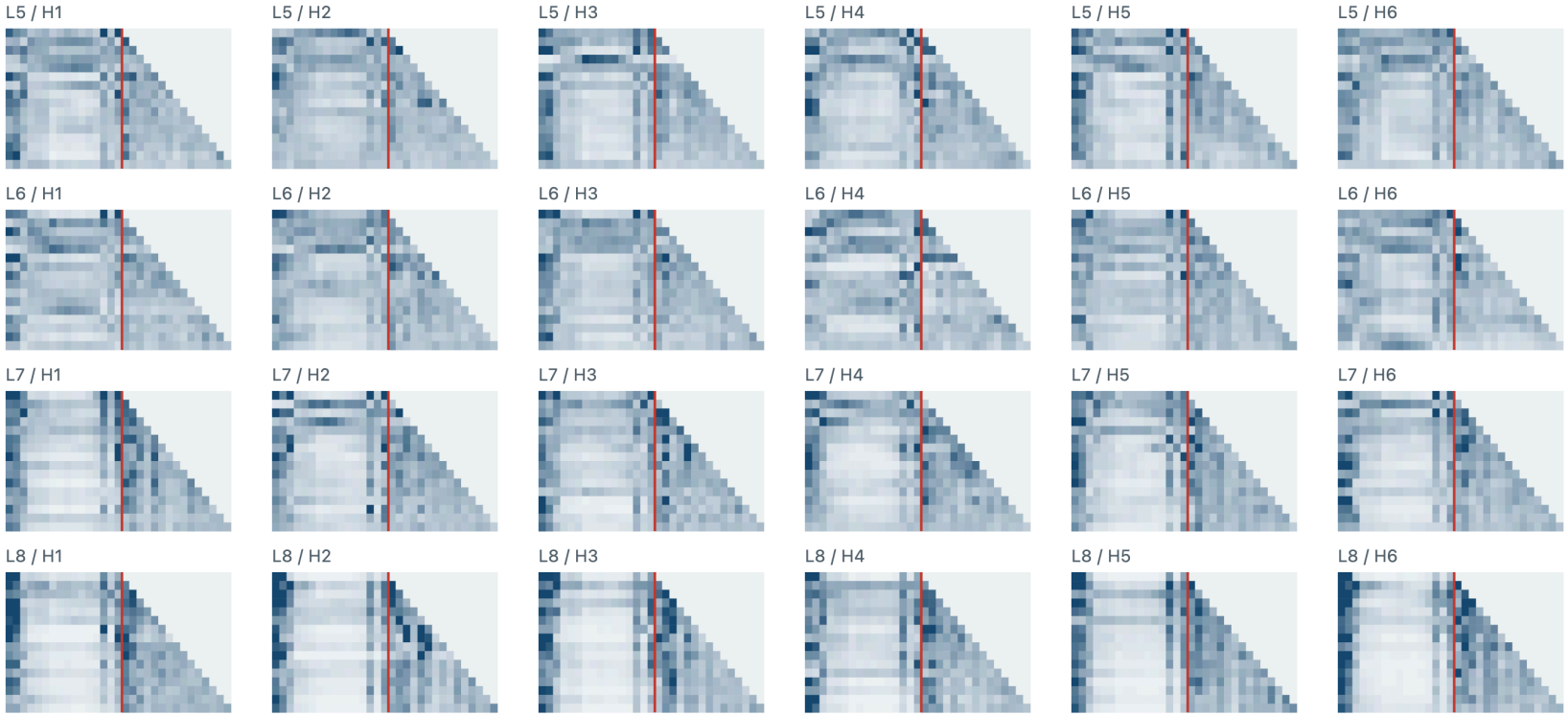
同じ指示：**各要素を絶対値にして、降順に並べる**



生成結果：絶対値化と降順整列のコード生成

15M：attention の可視化（L5〜L8）

同じ指示：**各要素を絶対値にして、降順に並べる**



生成結果：絶対値化と降順整列のコード生成

指示情報の遮断による合格数の低下

学習時に準拠した文型による24操作の診断

モデル	通常の生成	指示の情報を止めた生成
1M・1 epoch	3 / 24	**0 / 24**
1M・3 epoch	19 / 24	**0 / 24**
5M・1 epoch	22 / 24	**0 / 24**
15M・1 epoch	22 / 24	**2 / 24**

操作の選択に対する指示情報の寄与

表現を変えた場合の性能評価

調べること	今回分かったこと
決めた文型での動作	1M・3 epochでも高い全体合格率
未学習の言い換え	1M・3 epochと15M・1 epochは15 / 30
指示の冒頭の変更	1M・3 epochは12→9 ／ 5M・1 epochは12→1（各14操作）

一種類のプロンプトでの精度に加え
表現を変えても性能を保てるかの評価の必要性

この結果が示す範囲

- 訓練：24操作から最大3操作を組み合わせるコード生成 ／ 追加評価：4操作までの評価
- 冒頭変更の比較：14操作に限定（日本語能力全体を対象としない測定）
- 学習：一つの乱数設定（繰り返し実験によるばらつきは未確認）

今後の課題：文型の偏りを減らした合成データによる表現への頑健性の再評価

Appendix D：Boku-nano 全体図

日本語の限定的な指示から solve(xs, k) を生成する小型Decoder-only Transformer
意味ASTを起点に共通の意味仕様で接続するデータ生成・学習・評価

1 意味

24の基本操作
抽出 10：偶数・奇数・>k・≥k・<k・≤k・kの倍数・正・負・ゼロ
変換 8：+k・−k・×k・2倍・3倍・符号反転・絶対値・二乗
並べ替え 3／切り出し 3

→

意味AST　12,720仕様
24操作からの順序付き1〜3操作の選択（重複なし）
24 ＋ 24×23 ＋ 24×23×22 ＝ 12,720
`{"sequence": [{"filter": ["even"]}, {"map": ["add_k"]}]}`

→

参照インタプリタ（期待値の計算）
意味ASTの入力リストへの適用と正しい出力の計算
operator・itertools・heapq・deque・while による生成solve関数の表現空間の外側での実装
→ 正解コードの検証（2）とhidden test採点（4）に利用

↓ 意味ASTの分割　　↓ 期待値で検証（参照インタプリタ → 正解コード生成）

2 データ

意味ASTの分割
組合せ汎化用：全12,720のうち670
残り 12,050 のハッシュ順での分割
　train　9,646（1操作24は全部）
　validation　1,202
　通常テスト　1,202（境界値も流用）
別枠：同一操作の反復 1,152
　　日本語言い換え 30指示

→

正解コード生成（自作テンプレート）
style_specによるfor／内包表記・変数名・スライス表記などの変更
277,440候補から全条件の通過例のみ採用
構文・Safe-AST・signature・入力非破壊・41入力での参照インタプリタとの出力一致

日本語指示生成
ルール：承認済み表現辞書（train 522件）を定型文へ連結
教師言い換え：Qwen3-4B-AWQ が意味と順序を保って言い換え

→

評価JSONL　108,590件
validation 24,040
正式5テスト 84,550問
hidden入力はコード作成時と別の入力

訓練JSONL　192,900件
1操作 460／2操作 8,680／3操作 183,760
ルール文 96,510＋言い換え 96,390
1意味ASTあたり原則20件

1レコードの中身
instruction_ja（日本語指示）
semantic_ast（意味AST）
reference_code（正解コード）
style_spec・code_style
split・tests・verification
SHA-256・seed・生成器version
教師モデル由来かどうかの来歴

↓ train の指示とコード（訓練JSONLから）

3 学習

BPEトークナイザ
ByteLevel BPE・語彙 2,048
trainの日本語指示と正解コードだけから学習（評価データの除外）
min_frequency 5・最大token長 24
unknown token 0件

→

学習系列とloss
`<|bos|><|task|> {instruction_ja}`
`<|code|> {reference_code}<|eos|>`
lossの対象はコードとEOSのみ（指示は条件としての参照のみ）
batch 512件・1エポック 377 step・AdamW・bf16

→

Decoder-only Transformer（ランダム初期値から学習）
RoPE・RMSNorm・SwiGLU・通常のMulti-Head Attention・最大系列長 256
　1M　d=128・3層・4 head
　5M　d=256・5層・4 head
　15M　d=384・8層・6 head
3つの規模をそれぞれ1エポック学習（RTX 5090）

↓ 学習済みモデル　　↓

4 評価

hidden test評価（正式5テスト 84,550問）
greedy生成 → 構文 → Safe-AST → signature → 隔離実行
→ hidden入力すべてで参照インタプリタと同じ出力なら合格
文字列一致ではなく実行結果による判定
　通常 24,040　　組合せ汎化 13,400　　日本語言い換え 30
　同一操作の反復 23,040　　境界値 24,040

→

主な結果（1エポック・正式5テスト合計）
15M　99.56%（84,180 / 84,550）
5M　96.46%（81,557 / 84,550）
1M　1.74%（1,467 / 84,550）

内部解析
Q・K・Vを取り出しattentionを計算
head別のattention配分・Value norm・
BOS sink・attention rollout
head／位置ablation：NLL差（nat）と
再生成コードの合格数で因果の確認

5 デモ

自由な日本語
例：全部プラスにして大きい順に並べて

→

Qwen3-0.6B（ブラウザ）
自由文からCNLへの翻訳のみ
追加学習なし・コード生成なし

→

JavaScript検査
24操作・最大3操作・
定型文法に合うか確認

→

Boku1-nano ONNX
学習済みモデルを選んで
ブラウザ内でgreedy生成

→

Pythonコード
`def solve(xs, k) -> list[int]`

自作ルール・決定的な処理　Qwenを使う処理（日本語のみ）　データ・出力　学習・モデル　評価・解析　↓ 参照インタプリタによる検証

参考文献

Eldan & Li (2023)
TinyStories: How Small Can Language Models Be and Still Speak Coherent English?

Finke et al. (2025)
Parameterized Synthetic Text Generation with SimpleStories

Maini et al. (2024)
Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling

Yu & Xiong (2026), COLM
Generating Pretraining Tokens from Organic Data for Data-Bound Scaling (SynPro)

Gunasekar et al. (2023)
Textbooks Are All You Need

Liu et al. (2023)
TinyGSM: achieving >80% on GSM8k with small language models

Kim et al. (2026), COLM
Data-efficient pre-training by scaling synthetic megadocs

Ouyang et al. (2022)
Training language models to follow instructions with human feedback

Zheng et al. (2023)
Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

Lambert et al. (2024)
Tulu 3: Pushing Frontiers in Open Language Model Post-Training

Gan & Mori (2023)
Sensitivity and Robustness of Large Language Models to Prompt Template in Japanese Text Classification Tasks

Takayama et al. (2026)
Evaluating the Effect of Question Wording Variations on Answer Consistency in Large Language Models